📄 ER 모델 연습 2

Scenario 2 — 게임 유저 관리 시스템

---

1. Scenario 이해

게임 시스템에서 유저는 계정을 가지고 캐릭터를 생성합니다.
캐릭터는 아이템을 가질 수 있습니다.

---

2. Entity 도출

✏️ Answer:

1. User(유저)
2. Character(캐릭터)
3. Item(아이템)
4. Inventory(아이템 목록)

---

3. 개념 구분

유저(User)와 캐릭터(Character)는 어떻게 다른가?

✏️ Answer: 아니요 한 유저는 여러 캐릭터를 가질 수 있습니다.

---

4. Attributes 정의

User

Attributes:
- user_id
- email
- password
PK: **user_id**

Character

Attributes:
- character_id
- name
- level

- [stats]
- class / type
- user_id (FK)
PK: **character_id**

Item

Attributes:
- **item_id**
- name
- type
PK: **item_id**

캐릭터와 아이템 (M:N) ⇔ 연결 가능한 개체 필요 => Inventory 객체 생성하여 매핑

---

5. M:N 관계 해결

캐릭터와 아이템 사이의 관계는?

✏️ Answer:

Type: M:N
Reason: 한 캐릭터는 여러 아이템을 가질 수 있고, 하나의 아이템은 여러 캐릭터가 가질 수 있기 때문

Inventory

Entity Name: Inventory
Attributes:
- character_id (FK)
- item_id (FK)
- quantity
PK: 복합 키**: (character_id, item_id)**

---

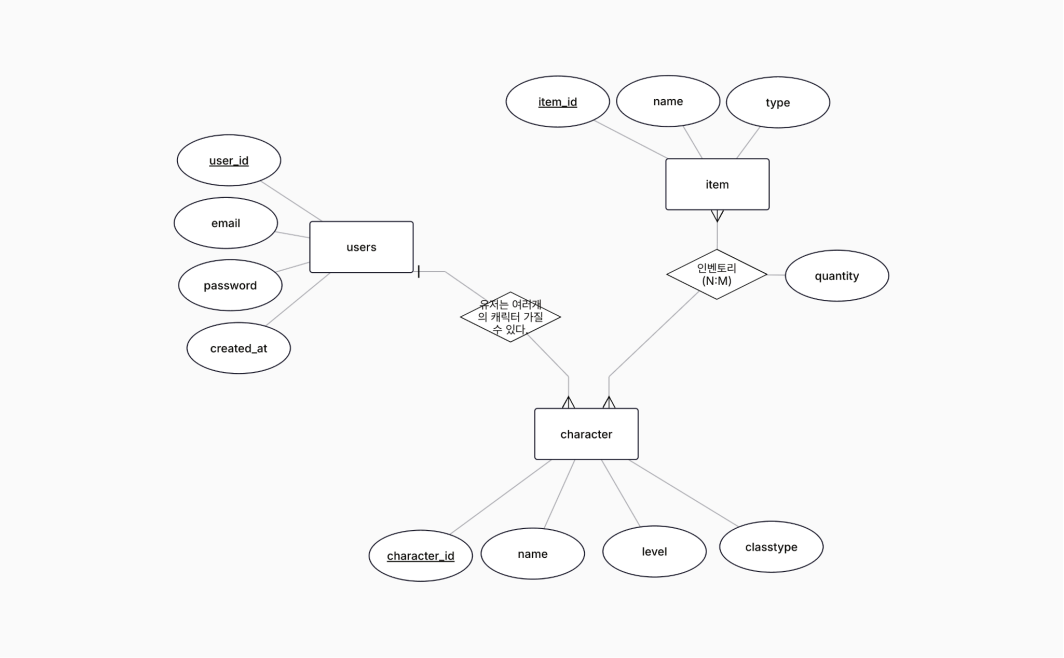
6. 관계 정의

유저 ⇔ 캐릭터(1:n)
캐릭터 ⇔ 아이템목록 (1:n)
아이템 ⇔ 아이템목록 (1:n)

카디널리티
User - Character | (1,1) : (0,N)
Character - Inventory | (1,1) : (0,N)
Item - Inventory | (1,1) : (0,N)

---

7. ER 다이어그램 만들기



---

8. SQL 작성

- postgres

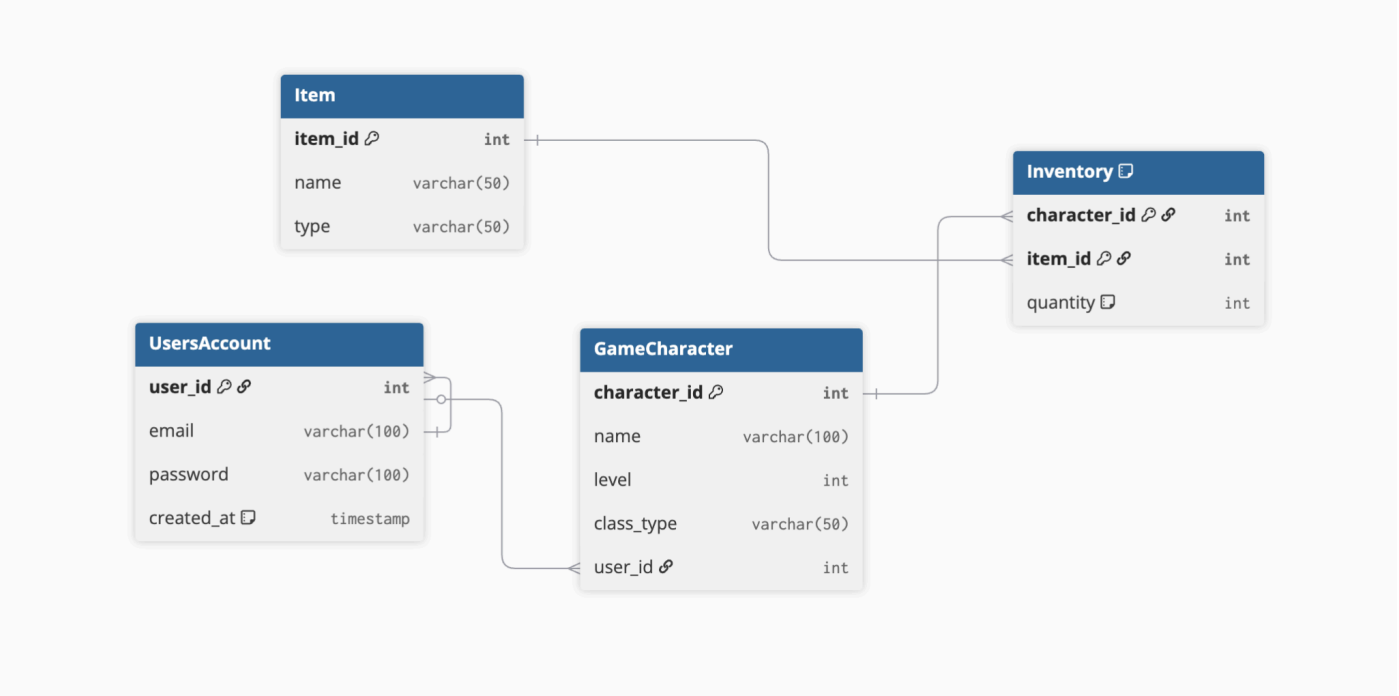
```
-- User
CREATE TABLE UsersAccount (
                            user_id INT PRIMARY KEY,
                            email VARCHAR(100),
                            password VARCHAR(100),
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Character (예약어 이슈 때문에 바꿈)
CREATE TABLE GameCharacter (
                             character_id INT PRIMARY KEY,
                             name VARCHAR(100),
                             level INT,
                             class_type VARCHAR(50),
                             user_id INT,

                             FOREIGN KEY (user_id) REFERENCES UsersAccount(user_id)
);

-- Item
CREATE TABLE Item (
                     item_id INT PRIMARY KEY,
                     name VARCHAR(50),
                     type VARCHAR(50)
);

-- Inventory (M:N 해결)
CREATE TABLE Inventory (
                         character_id INT,
```

```
                            item_id INT,
                            quantity INT DEFAULT 1,

                            PRIMARY KEY (character_id, item_id),

                            FOREIGN KEY (character_id) REFERENCES
GameCharacter(character_id),
                            FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

- dbdiagram

```
Table UsersAccount {
 user_id int [pk]
 email varchar(100)
 password varchar(100)
 created_at timestamp [default: `CURRENT_TIMESTAMP`]
}

Table GameCharacter {
 character_id int [pk]
 name varchar(100)
 level int
 class_type varchar(50)
 user_id int
}

Table Item {
 item_id int [pk]
 name varchar(50)
 type varchar(50)
}

Table Inventory {
 character_id int
 item_id int
 quantity int [default: 1]

 Indexes {
   (character_id, item_id) [pk]
 }
}

Ref: GameCharacter.user_id > UsersAccount.user_id
Ref: Inventory.character_id > GameCharacter.character_id
Ref: Inventory.item_id > Item.item_id
```

- Dbdiagram 스크린 샷



🧠 Reflection (중요)

---

1. 공통점

두 시나리오의 공통점은 무엇인가?

✏️ Answer:

두 시나리오 모두 M:N 관계가 존재하며, 이를 해결하기 위해 중간 엔티티를 생성해야 한다는 점

---

2. 핵심 개념

M:N 관계는 어떻게 해결하는가?

✏️ Answer:

M:N 관계는 직접 표현할 수 없기 때문에, 중간 테이블(예: Enrollment, Inventory)을 생성하여 1:N 관계 두 개로 분해한다

---

3. 가장 어려웠던 점

가장 어려웠던 부분은 무엇인가?

✏️ Answer:

M:N 관계를 발견하고 이를 새로운 엔티티로 변환하는 과정이 가장 어려웠다